TEAM GG

AI SOLUTION DEVELOPMENT PROJECT

PRESENTATION

TEAM GG

CONTRIBUTION

YIK HENG

- EDA for Marital Status, Personal Loan, Campaign Calls
- Pipeline structure & Nodes
- Presentation Slide
- Improve the model's performance & README.md

MATTHEW CHRISTOPHER

- EDA for Occupation, Housing Loan, Previous Contact Days & Overall EDA
- Github structure & README.md
- Logistic Regression
- Presentation Slide

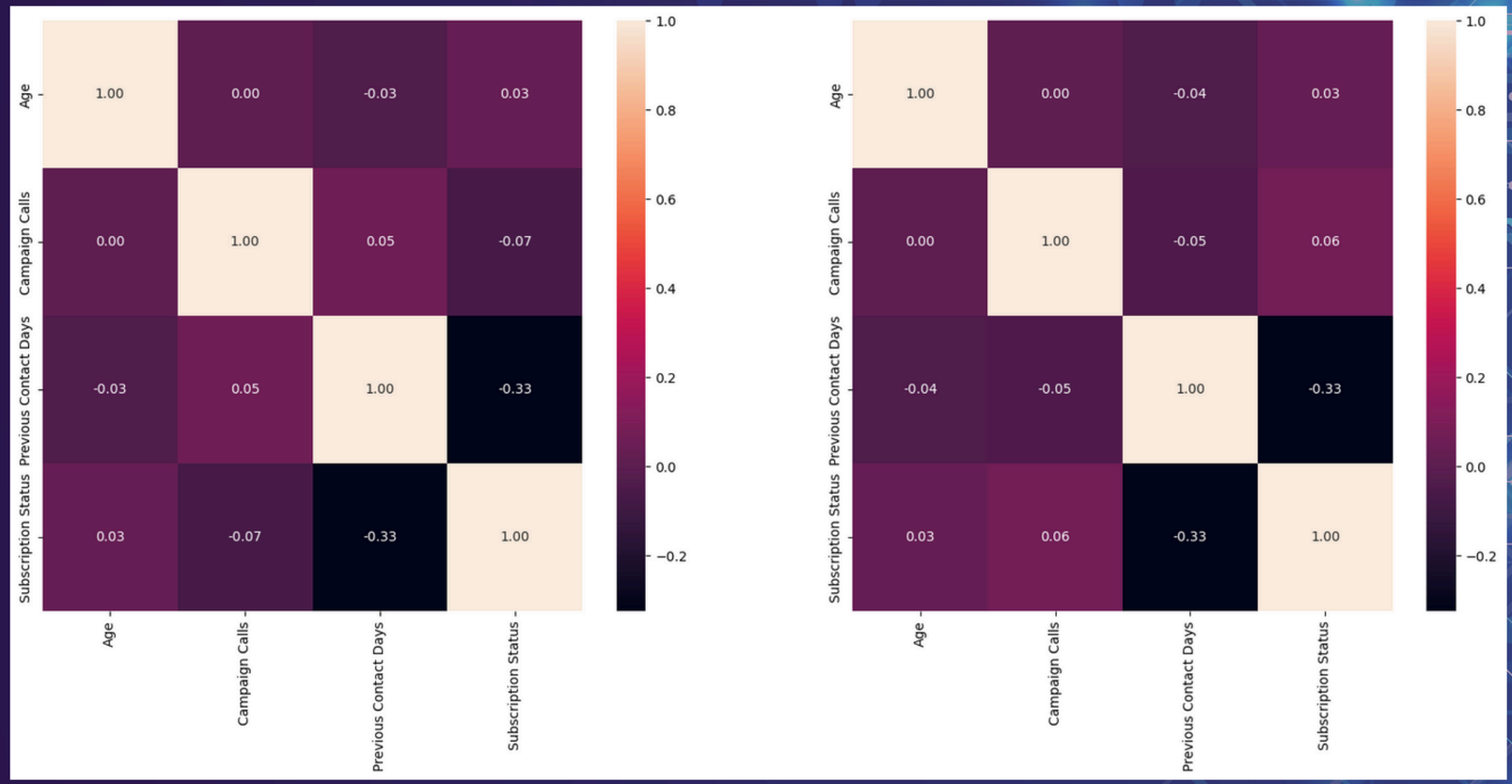
JADE WONG

- EDA for Age, Credit Default, Contact Method
- Pipeline structure, XGBoost Model
- requirements.txt, run.sh, Dockerfile, README.md
- Presentation Slide

TEAM GG

NULL/ERROR HANDLING METHODS

The campaign calls imputation and scaling



TEAM GG

VISUALIZATIONS

Subscription count vs Attributes

- Descriptive stats → detect outliers (age 150), understand distribution
- Age grouping + countplot → Adults subscribe most, Youth & Elderly lowest
- Countplot (Contact Method) → Cellular outperforms Telephone in subscription
- Correlation Heatmap → No strong linear predictors → features must be combined

```
median_age = df_eda2.loc[df_eda2["Age"] < 100, 'Age'].median()
df_eda2.loc[df_eda2["Age"] > 100, "Age"] = median_age
```

✓ 0.0s

```
median_age
```

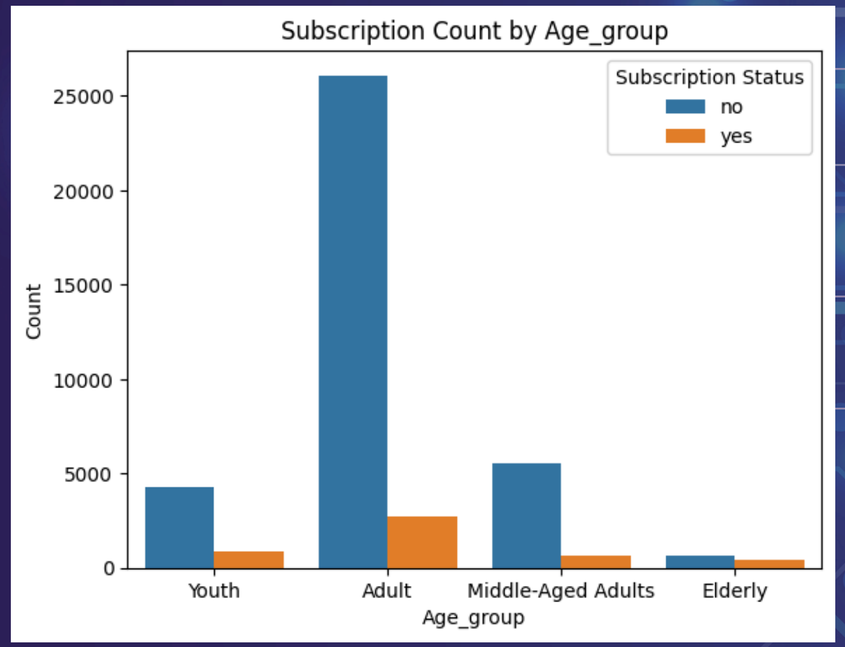
✓ 0.0s

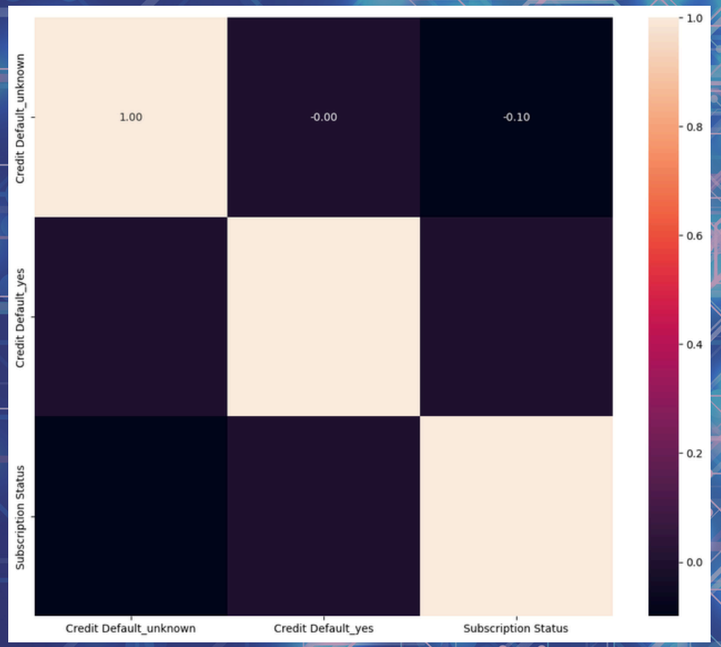
```
np.float64(38.0)
```

```
bins = [0, 29, 49, 59, 100]
labels = ['Youth', 'Adult','Middle-Aged Adults','Elderly']

df_eda2["Age_group"] = pd.cut(df_eda2['Age'], bins = bins, labels = labels)
df_eda2[["Age","Age_group"]]
```

[94] ✓ 0.0s





TEAM GG

MODEL CHOICES

RANDOM FOREST

do not require as much data transformation

Reliable as a baseline model

CAT BOOST

Can handle imbalance datasets well. It can also adjust the weights of the classes.

HGBOOST

It follows random forest, except each tree tries to fix its previous mistakes.

It is highly tunable

TUNING METHODS

TEAM GG

```python
def random_search_catboost(X_train, y_train):

    catboost = CatBoostClassifier(
                                  loss_function="Logloss",
                                  eval_metric="F1",
                                  verbose=0,
                                  task_type="CPU",
                                  random_seed=42,
                                  class_weights=[1, 8]
                                 )

    param_dist = {
        "iterations": np.arange(300, 1500, 50),
        "depth": np.arange(3, 10),
        "learning_rate": np.linspace(0.01, 0.3, 10),
        "l2_leaf_reg": np.linspace(1, 10, 10),
        "bagging_temperature": np.linspace(0, 1, 10),
        "random_strength": np.linspace(1, 20, 10),
        "border_count": np.arange(32, 256, 32)
    }

    f2 = make_scorer(fbeta_score, beta=2)

    search = RandomizedSearchCV(
        catboost,
        param_dist,
        n_iter=80,
        scoring=f2,
        cv=5,
        verbose=1,
        n_jobs=-1,
        random_state=42
    )

    search.fit(X_train, y_train)

    logger = logging.getLogger(__name__)

    # Log best parameters + score
    logger.info(f"Best CatBoost Params: {search.best_params_}")
    logger.info(f"Best CV F2 Score: {search.best_score_}")

    return search.best_estimator_
```

```python
def grid_search_random_forest(X_train: pd.DataFrame, y_train: pd.Series) -> RandomForestClassifier:
    """Performs grid search to find the best hyperparameters for Random Forest classifier."""
    clf = RandomForestClassifier(random_state=42, n_jobs=-1, class_weight='balanced')

    param_grid = {
        'n_estimators': [700, 750, 800],
        'max_depth': [8, 10, 12],
        'max_features': ['sqrt', 'log2'],
        'min_samples_split': [5, 7, 10],
        'min_samples_leaf': [1, 2, 4]
        }

    grid_search = GridSearchCV(estimator=clf, param_grid=param_grid, cv=5, n_jobs=-1, scoring='f1', verbose=2
    grid_search.fit(X_train, y_train)
    best_clf = grid_search.best_estimator_
    logger = logging.getLogger(__name__)
    logger.info("Best parameters found: %s", grid_search.best_params_)
    return best_clf
```

TEAM GG

EVALUATION METHODS

ACCURACY

- Focuses on the model's prediction 'correctness' on the dataset provided
- Good for learning from data
- Due to the class imbalance in the dataset, focusing on accuracy would lower the model's generalisation → overfitting

F1 SCORE

- Evaluates the balance of Precision and Recall
- Balances how **harshly** the model predicts, and how well the model **finds** the **positives**
- Overall precision is not the most important component, especially as the dataset is skewed towards non-subscribers

F2 SCORE

- Balances precision and recall, with a higher emphasis on recall
- By weighing recall twice as heavily as precision, evaluating for f2 helps find models that are better at **finding** the **target market**, even if it may produce more false positives

AI SOLUTION DEVELOPMENT PROJECT - TEAM GG

THANK YOU!

THANK YOU FOR EXPLORING PROGRAMMING WITH US.
LET'S INNOVATE TOGETHER!